Celer: A Lookup Argument for Large-Scale Queries

Wenjie Qu*[1], Yanpei Guo*[1], Zhen Xuan[1], Xuanming Liu[2], and Jiaheng Zhang[1]

[1] National University of Singapore
[2] Zhejiang University

Abstract. Lookup arguments are designed to prove that all elements in a query vector appear in a lookup table. These arguments are critical for efficiently proving ZK-unfriendly statements, such as range checks and the evaluation of complex functions. Most existing lookup arguments are optimized for scenarios where the query size is smaller than the table size. However, in many real-world applications, the query size m is often much larger than n, the table size. This disparity leads to inefficiencies in such schemes.

To overcome this limitation, we propose a novel lookup argument in which the prover's runtime increases slowly with m, thereby providing improved scalability. The efficiency of our lookup PIOP stems from reducing the number of commitments to be sublinear in the query size m, while maintaining field operations linear in m. Experimental results demonstrate that our lookup scheme significantly outperforms state-of-the-art lookup schemes. For the workload of query size $m = 2^{28}$ and table size $n = 2^{16}$ (a real world case in proving Llama language model inference), our scheme achieves a 25.9× prover-time speedup over plookup, 9.4× faster than LogUp, and 2.4× faster than logUp-GKR.

1 Introduction

Zero-Knowledge Succinct Non-interactive Arguments of Knowledge (zkSNARKs) [GWC19] are a class of zero-knowledge proof systems that enable succinct and non-interactive verifiable computations. Modern zkSNARKs constructions often follow a recipe that compiles a *Polynomial Interactive Oracle Proof* (PIOP) [BFS20] using a *Polynomial Commitment Scheme* (PCS) [KZG10,GLH+25,QGZ26], and finally applies the Fiat-Shamir transformation [FS86] to render it non-interactive. In a PIOP, the prover $\mathcal{P}$ and verifier $\mathcal{V}$ interact over multiple rounds, where each prover message is provided as oracle access to a polynomial that verifier can query at arbitrary evaluation points. In practice, these polynomial oracles are instantiated using a PCS, which allows the prover to convince the verifier of the correctness of polynomial evaluations with succinct proofs. Nowadays, zkSNARKs have attracted significant attention both in blockchain scalability [AST24] and in machine learning verification [LXZ21,QSL+25,QGYZ25].

* Equal contribution.

Lookup PIOPs [CBBZ23] are special PIOPs that enable $\mathcal{P}$ to convince $\mathcal{V}$ that all elements in a query vector $a \in \mathbb{F}^m$ appear in a public vector $T \in \mathbb{F}^n$, known as the table. Usually, $\mathcal{V}$ is given oracle access to query vector a in advance. Formally, $\mathcal{P}$ convinces $\mathcal{V}$ about the following relation:

$$\mathcal{R}_T = \{(\mathbb{x} = [[a]], \mathbb{w} = a) : \forall x \in \{0,1\}^{\log m}, a(x) \in T\} \tag{1}$$

where $[[a]]$ represents oracle of multilinear polynomial $a(\cdot)$. Lookup arguments [GW20] are obtained from lookup PIOPs by compiling the oracles with a PCS.

The main usage of lookup arguments in zkSNARKs is reducing the cost of proving non-arithmetic relations. A common use case is *range checking* [BBB+18], which involves checking that each element in the vector a falls within a specified range $[\ell, r]$. Range checks are widely employed in real-world applications, such as preventing transaction amount overflows in confidential transactions [SCG+14] and validating input ranges in robust federated learning [BGL+23,LBV+23]. A naive approach to range checking is to prove the correctness of the bit-decomposition of each element in a, but this incurs high overhead due to committing to multiple field elements per value. In contrast, lookup arguments offer a simple and more efficient alternative: one can prove that each element in a appears in a fixed table $T = \{\ell, \ell + 1, \cdots, r\}$.

Another key application of lookup arguments is proving non-linear functions like exponentiation (e.g., $y = 2^x$). For instance, several recent works [KHSS22,SLZ24,LWQ+24,HCL+24] adopted the lookup to prove the exponentiation and GeLU [HG16] function used in machine learning. For each function, a public lookup table is constructed containing all valid (x, y) pairs. By directly proving the pairs of inputs and the corresponding outputs appearing in the lookup table, the prover avoids the expensive arithmetization of complex non-linear functions.

Recent years have seen a variety of lookup arguments [GW20,Hab22,STW24], whose essential distinction lies in the design of their underlying lookup PIOP. The lookup relation (1) has an equivalent polynomial form $\prod_{i=1}^{m}(X - a_i) = \prod_{i=1}^{n}(X - t_i)^{e_i}$, where $\{e_i\}_{i\in[n]}$ denotes the number of times each element a_i appears in the table. The first lookup PIOP, plookup [GW20], tries to prove this polynomial equality by examining the sorted version of the concatenated query a and table T, achieving prover cost dominated by committing to $O(m + n)$ field elements. Since then, many advances [ZBK+22,EFG22,CFF+24] have been made on lookup arguments primarily focusing on optimizing the performance of scenarios where the table size n significantly exceeds the query size m [HACF25].

In contrast, when the query size m significantly exceeds the table size n, existing lookup arguments exhibit sub-optimal prover efficiency due to their differing design goals. For instance, although cq [EFG22] removes the dependence of prover complexity on the table size n, our experiments show that it is more expensive than the classical plookup [GW20] scheme when $m \gg n$. The essential reason behind this inefficiency is that most existing lookup arguments are bottlenecked by committing to $O(m)$ *internal witnesses* (often with a large constant factor) introduced by the underlying PIOP.

Nevertheless, many real-world applications feature the regime $m \gg n$. This arises because, in large-scale problems, the number of queries scales with the problem size, whereas table sizes often remain small to moderate. For example, in zero-knowledge machine learning (ZKML) systems [LXZ21,SLZ24,QSL+25,GQW+26], medium-precision quantization (e.g., 16-bit) is widely used to reduce prover costs, so a table of size 2^{16} usually suffices. In contrast, the query size tends to be very large due to the scale of ML models. In particular, zkLLM [SLZ24] (CCS'24) shows that a single inference of LLaMA-7B [TLI+23] on a 512-token input evaluates GeLU [HG16] more than 2×10^8 times, yielding $m \approx 2^{28}$ lookups—about three orders of magnitude larger than the table ($n = 2^{16}$).

Another example arises in the recently introduced verifiable image editing system Veritas [DCB24] (S&P'25). Veritas checks the integrity of various image editing operations, including the grayscale operation: $gr = \text{round}(\frac{30R+59G+11B}{100})$. Proving this operation requires querying a lookup table containing values from 0 to 99. For an image with 2 megapixels, the query size reaches $m = 2 \times 10^6$, as each pixel queries the table once. In this case, m exceeds the table size $n = 100$ by more than four orders of magnitude. Finally, in ZKLP [EZC+25] (S&P'25), the state-of-the-art proof system for floating-point computation, proving FP32 and FP64 computations requires querying two tables of size 256 and 64, respectively. The query size scales linearly with the number of floating-point additions and multiplications being proved, often exceeding 2^{20}.

Celer: lookup PIOP with sublinear commitment cost. Given that many real-world scenarios exhibit $m \gg n$, it is crucial to design new schemes that are well-suited for this setting. In this paper, we introduce Celer, a lookup PIOP designed to scale efficiently with a large number of lookup queries. Our design is motivated by the observation that the prover time of most existing lookup PIOPs is bottlenecked by the commitment to $O(m)$ internal witnesses. For example, in plookup [GW20], this bottleneck arises from committing to the sorted version of the concatenated query a and table T. In LogUp [Hab22] and its follow-up works such as cq [EFG22] and cq++ [CFF+24], it comes from committing to reciprocals of the form $\frac{1}{X-a_i}$. In Lasso [STW24], the $O(m)$ commitment is introduced by committing to read and write timestamps for memory checking.

The advantage of Celer comes from reducing the committed internal witness in the lookup PIOP to be *sublinear* in the query size m. The core advantage of Celer comes from the design strategy of directly proving the classical product relation corresponding to lookup, transforming the lookup PIOP's bottleneck into a single grand-product check on the query vector. Compiling Celer PIOP with PCS whose opening algorithm only requires sublinear group operations (e.g., Dory PCS [Lee21]), we can obtain a lookup argument with group operations sublinear in query m[3]. This lookup argument is highly efficient under scenarios where $m \gg n$. Given the widespread use of lookup arguments in SNARK constructions, our proposed lookup argument has the potential to significantly

[3] Existing lookup arguments [GW20,Hab22,STW24,EFG22] all treat the commitment of the query vector as a public input. Consequently, their analyses exclude the cost of committing to the queries. We follow this convention in this paper.

enhance the scalability of zkSNARKs, leading to substantial cost savings in practical deployments.

1.1 Our Contribution

In this paper, we present a novel and efficient lookup PIOP scheme, Celer, designed for the case $m \gg n$, i.e., large-scale queries to a moderate-size table in real-world ZKP systems. Our contributions are listed as follows:

- Celer is the first lookup PIOP designed for high efficiency under scenarios where $m \gg n$. Its core insight lies in directly proving the classical polynomial relation corresponding to lookup, transforming the lookup PIOP's bottleneck into a single grand-product check [SL20,CBBZ23,Tha13] of size m.
- An important component of Celer PIOP is an efficient protocol for *proving products of powers*, which eliminates the need to commit to $O(n \log m)$ field elements and to prove costly memory read–write operations. This protocol may also be of independent interest.
- We conduct experiments to evaluate Celer's performance. The results demonstrate that Celer achieves a substantial advantage over state-of-the-art lookup arguments. Under the workload of query size $m = 2^{28}$ and table size $n = 2^{16}$ (a real-world case in proving LLaMA-7B language model inference), Celer achieves a remarkable 25.9× speedup in prover time compared to plookup, outperforming LogUp by 9.4×, Lasso (EUROCRYPT'24) by 3.8×, and logUp-GKR by 2.4×.

By compiling Celer lookup PIOP with Dory PCS [Lee21], Celer lookup argument achieves sublinear group operations with respect to the query size m and field operations linear in m with a small constant factor. In contrast, most existing lookup arguments need at least $O(m)$ group operations, considering elliptic-curve-based PCS.

Celer achieves substantially faster prover times than existing lookups, as shown in Table 1, at the cost of larger proof size. This tradeoff is favorable under many scenarios.

1.2 Related work

Lookup arguments were first introduced by Arya [BCG+18] to prove that every element of a committed query vector $\{a_i\}_{i\in[m]}$ belongs to a table $\{t_i\}_{i\in[n]}$. Arya formulates lookup membership through multiplicities: it proves the existence of non-negative integers $\{e_i\}_{i\in[n]}$ such that

$$\prod_{i=1}^{m}(X - a_i) = \prod_{i=1}^{n}(X - t_i)^{e_i}\,. \tag{2}$$

In Arya, the value of RHS is proven using a bit-decomposition circuit.

Plookup [GW20] later proposed a different lookup PIOP based on sorting. Instead of explicitly proving the multiplicities in Equation (2), Plookup checks a

Table 1: Properties of existing lookup PIOPs. n denotes the table size, m denotes the query vector size.

Lookup PIOP	Prover work	
	Field elements to commit	Field operations
plookup [GW20]	$5m$	$O(m \log m)$
Caulk (CCS'22) [ZBK$^+$22]	$15m$	$O(m^2 + m \log n)$
Baloo [ZGK$^+$22]	$14m$	$O(m \log^2 n)$
LogUp [Hab22]	$m + 2n$	$12m + 12n$
cq [EFG22]	$8m$	$O(m \log m)$
logUp-GKR [PH23]	n	$43m + 43n$
cq++ (PKC'24) [CFF$^+$24]	$8m$	$O(m \log m)$
Locq (PKC'24) [ZSG24]	$7m$	$O(m \log m)$
Lasso (EUROCRYPT'24) [STW24]	$3m + \frac{m}{\log^3 m} + n$	$20m$
Celer	$\frac{m}{\log^3 m} + O(m^{\frac{1}{w}} + n)$	$10m + O(m^{\frac{1}{w}} + n)$

• In our paper, we assume $m \gg n$, where m, n are powers of 2.
• For Lasso [STW24], we consider the case of unstructured table.
• Verifier work and proof size depend on the underlying PCS and are therefore not reported here.

sorted version of the concatenation of the query vector a and the table T. This yields a conceptually simple and highly influential construction, but its prover cost is dominated by committing to $O(m + n)$ field elements.

LogUp [Hab22] returned to the multiplicity-based viewpoint of Arya. It leverages logarithmic derivatives to transform Equation 2 into:

$$\sum_{i=1}^{m} \frac{1}{X - a_i} = \sum_{i=1}^{n} \frac{e_i}{X - t_i} \tag{3}$$

Equation 3 has the form of a sum of reciprocals. LogUp [Hab22] commits to the reciprocals and checks Equation 3 leveraging the sumcheck protocol, achieving smaller constant factors compared to plookup [GW20].

Subsequent works [EFG22,CFF$^+$24,ZSG24] have further improved the prover efficiency of LogUp, mainly considering the setting of $n \gg m$. Among them, leveraging (relatively expensive) pre-processing, cq [EFG22] eliminated the complexity dependency of LogUp [Hab22] on table size n, achieving a prover work of committing to $O(m)$ field elements and $O(m \log m)$ $\mathbb{F}$. Furthermore, Locq [ZSG24] (PKC'24) and cq++ [CFF$^+$24] (PKC'24) optimized the proof size and verifier time on top of cq [EFG22]. However, most logarithmic derivative-based lookup PIOPs [Hab22,EFG22,CFF$^+$24,ZSG24] require committing to $O(m)$ field elements, due to committing to the reciprocals on the LHS of Equation 3. LogUp-GKR [PH23] avoids committing to $O(m)$ field elements by applying the GKR

protocol to prove the fraction identities, at the cost of large constant factors in field operations.

Recently, Lasso [STW24] (EUROCRYPT'24) proposed an efficient lookup PIOP for unstructured tables. Its idea is to formulate the lookup relation as $a = M \cdot T$, where $M \in \{0,1\}^{m \times n}$ is a sparse matrix. Though Lasso improves prover efficiency over prior works, it still requires committing to $O(m)$ internal witnesses representing read and write timestamps in memory-checking.

Moreover, Lasso extends its construction to support structured tables of massive size (e.g., 2^{64} or 2^{128}), which commonly appear in applications such as zero-knowledge virtual machines (ZKVMs) [AST24]. It does so by exploiting the structural decomposability of the table, decomposing queries to a large structured table into queries to smaller sub-tables. TaSSLE [Dor24] further showed that any lookup argument for unstructured tables could be transformed into one for large structured tables in a black-box manner by adding an extra witness commitment.

The key properties of existing lookup PIOPs are shown in Table 1.

2 Preliminary

2.1 Notations

Let λ and $\mathsf{negl}(\lambda)$ be the security parameter and a negligible function, respectively. We denote a prime field $\mathbb{F}_p = \mathbb{F}$, cyclic groups $\mathbb{G}, \mathbb{G}_T$ of order p, a bilinear map $e : \mathbb{G} \times \mathbb{G} \to \mathbb{G}_T$, and a generator $g \in \mathbb{G}$ such that $e(g,g)$ generates $\mathbb{G}_T$. The generation of the bilinear map parameters is denoted by $\mathrm{bp} = (p, \mathbb{G}, \mathbb{G}_T, e, g) \leftarrow \mathrm{BilGen}\left(1^\lambda\right)$. We represent all groups using addition. When describing cost, N $\mathbb{F}$ denotes conducting N field multiplications. N $\mathbb{G}$ refers to a Multi-Scalar Multiplication (MSM)[4] of size N with arbitrary field elements. N $\mathbb{G}^*$ refers to a Multi-Scalar Multiplication (MSM) of size N with "small" field elements (in the range of $[0, 2^{32}-1]$). $\mathbb{P}$ refers to pairing operations.

For $n \in \mathbb{N}$, we denote $\{1, \cdots, n\}$ by $[n]$. By default, the indices of vectors, matrices, and tensors start at 1 unless stated otherwise. For any two vectors v_1, v_2, we denote their concatenation by $v_1 || v_2$. We denote the natural injection mapping an integer to its binary representation by $bin(\cdot)$. We denote a uniformly random sample from a set S by $\leftarrow_\$ S$. A relation is a set of pairs $(\mathtt{x}, \mathtt{w})$.

2.2 Multilinear extension

We define the identity function $eq(x,y)$, which takes two bit-strings x and y of the same length as input. The function $eq(\cdot,\cdot)$ returns 1 if $x = y$, and 0 otherwise. Suppose $\tilde{eq}(\cdot,\cdot)$ is the multilinear extension of $eq(\cdot,\cdot)$. Then, $\tilde{eq}(x,y) = \prod_{i=1}^{\ell} ((1-x_i)(1-y_i) + x_i y_i)$. We present the definition of the Multilinear Extension as follows:

[4] An MSM computes $\sum_{i=1}^{n} b_i \cdot A_i$, where $A_i \in \mathbb{G}$ and $b_i \in \mathbb{F}$.

Definition 1 (Multilinear Extension [CMT12]). *Let $V : \{0,1\}^\ell \to \mathbb{F}$ be a function. The multilinear extension of V is the unique polynomial $\tilde{V} : \mathbb{F}^\ell \to \mathbb{F}$ such that $\tilde{V}(x_1, x_2, \ldots, x_\ell) = V(x_1, x_2, \ldots, x_\ell)$ for all $x_1, x_2, \ldots, x_\ell \in \{0,1\}^\ell$. $\tilde{V}$ can be expressed as*

$$\tilde{V}(x_1, x_2, \ldots, x_\ell) = \sum_{b \in \{0,1\}^\ell} \tilde{eq}(x, b) \cdot V(b).$$

For a vector Z of length 2^ℓ, its multilinear extension $\tilde{Z}$ is defined as: $\tilde{Z}(x_1, \cdots, x_\ell) = \sum_{b \in [2^\ell]} \tilde{eq}(x, bin(b-1)) \cdot Z_b$.

2.3 Grand product check

Next, we introduce the concept of the grand product relation [SL20], which is commonly used in checking permutation relations [GWC19] and multi-set equalities [Set20,STW24]. This relation is a crucial component in the design of Celer.

Definition 2. *The grand product relation is defined as:*

$$\mathcal{R}_{\text{GP}} = \left\{ (v \in \mathbb{F}, com; f \in \mathbb{F}^m) : \begin{array}{r} v = \prod_{i=1}^{m} f_i \\ com = \mathsf{PC.Commit}(\mathsf{pp}, \tilde{f}) \end{array} \right\}$$

The argument proving the grand product relation is known as the "grand product check" [Tha13,SL20]. Here, we primarily introduce the protocol from Quark [SL20], which offers a good balance between prover time and proof size.

Consider a layered circuit that computes the grand product $\prod_{i=1}^{m} K_i$, consisting of a binary tree of multiplication gates with depth $\log m$. Let $V_i : \{0,1\}^{s_i} \to \mathbb{F}$ be a function that takes a binary string $b \in \{0,1\}^{s_i}$ as input and returns the output of gate b in layer i of the circuit. The gates are labeled such that, for any gate $p \in \{0,1\}^{s_i}$ in layer i, $V_i(p) = V_{i-1}(p||0) \cdot V_{i-1}(p||1)$.

Quark [SL20] packs the circuit gates of the last $\log m - 3\log\log m$ layers into a whole vector and checks these gates in a single sumcheck. Define the packed function $F : \{0,1\}^{\log m - 3\log\log m + 1} \to \mathbb{F}$. The gate values in the top most packed layer is stored as $F(0, x) = V_{3\log\log m}(x)$, for $x \in \{0,1\}^{\log m - 3\log\log m}$. For the subsequent layers, $F(1^\ell, 0, x) = V_{3\log\log m + \ell}(x)$, for $\ell \in [1, \log m - 3\log\log m - 1]$ and $x \in \{0,1\}^{\log m - 3\log\log m - \ell}$. The grand product result is stored in $F(1^{\log m - 3\log\log m}, 0) = V_{\log m}(0) = v$.

Based on the above construction, for $\ell \in [0, \log m - 3\log\log m - 1]$, $\forall x \in \{0,1\}^{\log m - 3\log\log m - \ell - 1}$, we have:

$$\begin{aligned} F(1^{\ell+1}, 0, x) &= V_{3\log\log m + \ell + 1}(x) \\ F(1^\ell, 0, x, 0) &= V_{3\log\log m + \ell}(x, 0) \\ F(1^\ell, 0, x, 1) &= V_{3\log\log m + \ell}(x, 1) \end{aligned}$$

Recall that $V_i(p) = V_{i-1}(p||0) \cdot V_{i-1}(p||1)$, we have to check $F(1^{\ell+1}, 0, x) \stackrel{?}{=} F(1^{\ell}, 0, x, 0) \cdot F(1^{\ell}, 0, x, 1)$ for $\ell \in [0, \log m - 3\log\log m - 1]$. Denote vector $y = 1^{\ell}||0||x$, by setting $F(1^{\log m - 3\log\log m + 1}) = 0$, checking the consistency of the last $\log m - 3\log\log m$ circuit layers could be transformed into checking $F(1, y) \stackrel{?}{=} F(y, 0) \cdot F(y, 1)$, $y \in \{0,1\}^{\log m - 3\log\log m}$.

The overall grand product check protocol is as follows: First, $\mathcal{P}$ commits to $\tilde{F}$, and proves that $\tilde{F}(1^{\log m - 3\log\log m}, 0) = v$ by opening the commitment on $\tilde{F}$. Next, $\mathcal{V}$ sends a random challenge $\beta \in \mathbb{F}^{\log m - 3\log\log m}$, and $\mathcal{P}$ uses the sumcheck protocol to prove that:

$$\sum_{y \in \{0,1\}^{\log m - 3\log\log m}} \tilde{eq}(\beta, y)(F(1, y) - F(y, 0) \cdot F(y, 1)) = 0$$

Then, $\mathcal{V}$ sends another random challenge $\delta \in \mathbb{F}^{\log m - 3\log\log m}$. $\mathcal{P}$ responds with a claim on $\tilde{V}_{3\log\log m}(\delta)$ and proves $\tilde{V}_{3\log\log m}(\delta) = \tilde{F}(0, \delta)$.

Starting from the claim on $\tilde{V}_{3\log\log m}(\delta)$, for each layer $i = 3\log\log m, \ldots, 1$, $\mathcal{P}$ applies the sumcheck protocol to prove that $\tilde{V}_i(z) = \sum_{p \in \{0,1\}^{s_i}} \tilde{eq}(z, p)\tilde{V}_{i-1}(p||0) \cdot \tilde{V}_{i-1}(p||1)$, thereby reducing the claim on $\tilde{V}_i(\cdot)$ to a claim on $\tilde{V}_{i-1}(\cdot)$. This process continues recursively until reaching a claim on $\tilde{V}_0(\cdot)$, which can be verified by opening the commitment to $\tilde{K}$.

The full protocol incurs a prover cost of $\frac{m}{\log^3 m}\ \mathbb{G} + O(m)\ \mathbb{F}$, a verifier cost of $O(\log m \log\log m)\ \mathbb{F}$, and a proof size of $O(\log m \log\log m)$. This cost analysis excludes the overhead of PCS openings, which depends on the underlying PCS.

3 Technical Overview

In this section we provide a high-level overview of Celer. Formally speaking, given the oracle access to a multilinear polynomial encoded query vector $a \in \mathbb{F}^m$, and public table $T \in \mathbb{F}^n$, Celer PIOP proves that $\{a_i\}_{i \in [m]} \subseteq \{T_i\}_{i \in [n]}$. The design of Celer starts from an observation in Arya [BCG+18] and LogUp [Hab22]: the lookup relationship is equivalent to certifying the existence of integers $e_1, \cdots, e_n \in [0, m]$, such that the polynomials $h(X) = \prod_{i=1}^{m}(X - a_i)$ and $g(X) = \prod_{i=1}^{n}(X - T_i)^{e_i}$ are identical. LogUp avoids the cost of proving expensive power operations in $g(X)$ by introducing the *logarithmic derivative* method, transforming the relation into a sum of reciprocals: $\sum_{i=1}^{m} \frac{1}{X - a_i} = \sum_{i=1}^{n} \frac{e_i}{X - T_i}$. However, in this work, we make a crucial observation that logarithmic derivative-based lookup PIOPs [Hab22,EFG22,CFF+24] incur high prover costs in scenarios where $m \gg n$, due to the need to commit to m reciprocals of the form $\frac{1}{X - a_i}$.

Instead, Celer avoids this overhead by directly proving the original polynomial equivalence between $g(X)$ and $h(X)$, which reduces to checking their evaluations at a random point r. The core challenge in Celer's design lies in efficiently proving the correctness of $g(r)$ and $h(r)$.

3.1 Proving $g(r)$

We first consider efficiently proving $g(r) = \prod_{i=1}^{n}(r - T_i)^{e_i}$, which is a product of powers.

Proof of $g(r)$ based on binary exponentiation. A straightforward way to prove the correctness of $g(r)$ is to use binary exponentiation (as adopted in Arya [BCG+18]). In plaintext computation, each term $(r - T_i)^{e_i}$ can be evaluated by decomposing e_i into binary form and first computing the sequence $(r-T_i), (r-T_i)^2, (r-T_i)^4, \ldots, (r-T_i)^{2^{\lfloor \log m \rfloor}}$ via repeated squaring. The desired power $(r - T_i)^{e_i}$ is then obtained by multiplying the selected powers according to the binary expansion of e_i. Similarly, when proving $g(r) = \prod_{i=1}^{n}(r-T_i)^{e_i}$, we can decompose e_i into bits and commit to $\{(r-T_i)^1, (r-T_i)^2, (r-T_i)^4, \cdots, (r-T_i)^{2^{\lfloor \log m \rfloor}}\}$, then $g(r)$ can be represented in an arithmetic circuit and proven by the GKR protocol [GKR15]. However, this solution requires committing to $O(n \log m)$ field elements, which is suboptimal asymptotically, as many existing lookup PIOPs [Hab22,STW24] achieve prover time linear in n.

(Generalized) Pippenger algorithm. Our novel observation for solving this challenge is that the computation of product of powers in $g(r)$ resembles the structure of a MSM. The *Pippenger algorithm* [Pip76] is the classical method for efficiently computing MSM. As noted in [Ber02], the algorithm can also be generalized to efficiently compute products of powers. Inspired by this, our starting point is to design a protocol that proves the correct execution of computing $g(r)$ with the Pippenger algorithm, thereby establishing its correctness.

We begin by reviewing how the Pippenger algorithm can be applied to compute a product of powers of the form $Y = \prod_{i=1}^{n} x_i^{e_i}$, where $x_i \in \mathbb{F}$ and $e_i \in [0, m]$. In the algorithm, each exponent e_i is first decomposed into w base-s digits such that $e_i = \sum_{j=1}^{w} s^{j-1} \cdot E_{j,i}$, where $E_{j,i} \in [0, s-1]$. This decomposition allows us to rewrite the expression of Y as:

$$Y = \prod_{i=1}^{n} x_i^{e_i} = \prod_{j=1}^{w}(\prod_{i=1}^{n} x_i^{E_{j,i}})^{s^{j-1}} \tag{4}$$

The task of computing Y is now reduced to computing intermediate products $R_j = \prod_{i=1}^{n} x_i^{E_{j,i}}$ for each $j \in [1, w]$, where each R_j is itself a product of powers, but with much smaller exponent values bounded by $s-1$. To compute R_j efficiently, terms with the same exponent are grouped into buckets. Specifically, for each $v \in [0, s-1]$, we define the bucket value $C_{j,v} = \prod_{E_{j,i}=v} x_i$, so that $R_j = \prod_{v=1}^{s-1}(C_{j,v})^v$. This expression can be further optimized as:

$$\prod_{v=1}^{s-1}(C_{j,v})^v = (C_{j,1} \cdots C_{j,s-1}) \cdot (C_{j,2} \cdots C_{j,s-1}) \cdots C_{j,s-1} \tag{5}$$

Based on the above relationship, we compute the suffix product vector S_j, where $S_{j,v} = \prod_{\ell=v}^{s-1} C_{j,\ell}$, and then compute $R_j = \prod_{v=1}^{s-1} S_{j,v}$.

In summary, the procedure of the Pippenger algorithm consists of the following steps: (1) Compute the bucket values $C_{j,v}$ based on x and E_j. (2) Compute

Algorithm 1 Compute $g(r) = \prod_{i=1}^{n}(r - T_i)^{e_i}$ via Pippenger

Require: Field element $r \in \mathbb{F}$, table $T \in \mathbb{F}^n$, exponents $e_1, \cdots, e_n \in [0, m]$, number of digits w. Base $s \leftarrow \lceil (m+1)^{\frac{1}{w}} \rceil$.
Ensure: $Y = \prod_{i=1}^{n}(r - T_i)^{e_i}$

Decompose $e_i = \sum_{j=1}^{w} E_{j,i} \cdot s^{j-1}$.
Initialize $C_{j,i} \leftarrow 1, S_{j,i} \leftarrow 1, j \in [1, w], i \in [0, s-1]$
➢ **Step (1): Compute bucket values**
for $j \leftarrow 1 \ldots w$ **do**
    for $i \leftarrow 1 \ldots n$ **do**
        $C_{j,E_{j,i}} \leftarrow C_{j,E_{j,i}} \cdot (r - T_i)$
    end for
end for
➢ **Step (2): Compute suffix product**
for $j \leftarrow 1 \ldots w$ **do**
    $S_{js-1} \leftarrow C_{js-1}$
    for $v \leftarrow s-2$ down to 1 **do**
        $S_{j,v} \leftarrow S_{j,v+1} \cdot C_{j,v}$
    end for
end for
➢ **Step (3): Final aggregation**
$Y \leftarrow 1$
for $j \leftarrow 1$ to w **do**
    $R_j \leftarrow 1$
    for $v \leftarrow 1$ to $s-1$ **do**
        $R_j \leftarrow R_j \cdot S_{j,v}$
    end for
end for
for $j \leftarrow 1$ to w **do**
    $Y \leftarrow Y \cdot R_j^{s^{j-1}}$
end for
return Y

the suffix products $S_{j,v}$ from the bucket values C_j. (3) Compute R_j from S_j, and aggregate all R_j to obtain Y. Since w is typically a small constant (e.g., $w = 2$), the overall computation requires only $O(n + m^{\frac{1}{w}})$ multiplications, which is significantly more efficient than the naive approach by binary exponentiation that requires $O(n \log m)$ multiplications.

Specialized proof for $g(r)$. To address this issue, we design a specialized and more efficient protocol for proving $g(r)$. The key idea is to avoid explicitly maintaining each bucket value $C_{j,v}$. Updating the bucket values requires frequent memory read and write operations, which are inherently ZK-unfriendly.

To bypass proving C_j, we encode the information of digit vector E_j into vector U_j and p_j, and prove the correctness of $S_{j,v}$ based on U_j and p_j. The encoding procedure is as follows: For each bucket value $C_{j,v}$, we enumerate over all of the T_i's contributing to this bucket value (all T_i such that $E_{j,i} = v$), then we concatenate all of these values into a sequence U_j by the order of v. We use

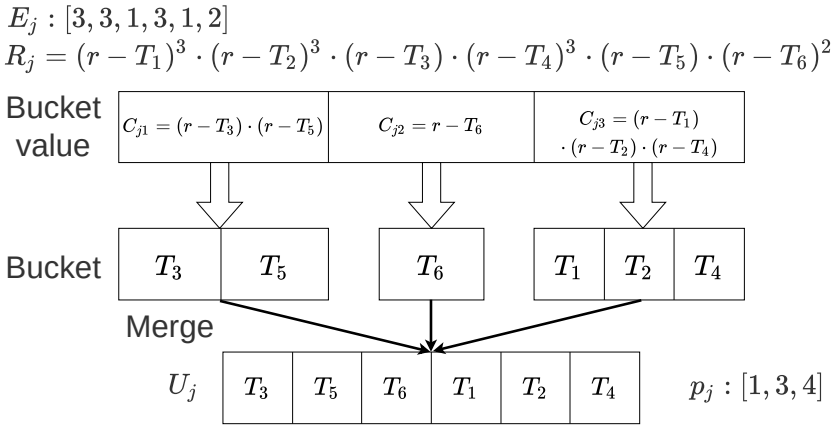


Fig. 1: Encoding the information of E_j into U_j and p_j.

p_j to mark the segments formed by different buckets, $p_{j,v}$ is the smallest element index in U_j corresponding to $E_{j,i} \geq v$, $v = 1, \cdots, s-1$. Note that the values $p_{j,v}$ may not be distinct across different v. Importantly, each $S_{j,v}$ can be efficiently derived from the sequence U_j using these indices.

Figure 1 gives an example of the encoding of U_j and p_j. We now illustrate the computation of S_j based on this encoding. Recall that $S_{j,v} = \prod_{\ell=v}^{s-1} C_{j,\ell}$, assume we want to compute $S_{j,2} = C_{j,2} \cdot C_{j,3} = (r - T_6) \cdot (r - T_1) \cdot (r - T_2) \cdot (r - T_4)$. We observe that $[r - T_6, r - T_1, r - T_2, r - T_4]$ is exactly a suffix of $r - U_j = [r - T_3, r - T_5, r - T_6, r - T_1, r - T_2, r - T_4]$ starting from index 3. Thus, $S_{j,2}$ can be obtained by the suffix product of $r - U_j$ starting from index 3. This is not a coincidence, because, by our design, each T_i appears in the order of its corresponding $E_{j,i}$ in U_j. The set formed by elements in U_j starting from index 3 is equivalent to the set of all T_i with $E_{j,i} \geq 2$. Similarly, the set of all T_i with $E_{j,i} \geq 3$ can be found at elements in U_j starting from index 4. We can conclude from our example that, the value of $S_{j,v}$ appears in the suffix product vector of $r - U_j$ at index $p_{j,v}$.

U_j and p_j now serve as ZKP-friendly encodings of E_j for proving R_j. Instead of directly committing to E_j, the prover commits to U_j and p_j as the witness. The correctness of S_j is established by proving that $S_{j,v} = \prod_{\ell=p_{j,v}}^{n} (r - U_{j,\ell})$ for each $v = 1, \cdots, s-1$ combined with an indexed lookup argument to "select" the suffix product values with indices p_{jv}. Notably, this lookup involves a much smaller query size $s - 1$ and table size n. Therefore, we adopt LogUp [Hab22], as the small query size ensures that the cost of this step does not dominate the overall cost of the Celer protocol. Proving that $R_j = \prod_{v=1}^{s-1} S_{j,v}$ is also standard. In total, the prover cost for proving $g(r)$ is committing to $O(n + m^{\frac{1}{w}})$ field elements and performing $O(n + m^{\frac{1}{w}})$ $\mathbb{F}$.

3.2 Proving $h(r)$

Our next step is to prove $h(r) = \prod_{i=1}^{m}(r - a_i)$. Given oracle access to $\tilde{a}$, the correctness of $h(r)$ can be proven by the grand product check protocol [SL20,CBBZ23,Tha13] presented in Section 2.3. It requires committing to $\frac{m}{\log^3 m}$ field elements and $O(m)$ inexpensive field operations. After invoking the protocol, the prover reduces the claim about $h(r)$ to a claim about a random evaluation of $\tilde{a}$.

Up to now, we have finished the construction of Celer PIOP. It achieves a prover work of committing to $\frac{m}{\log^3 m} + O(m^{\frac{1}{w}} + n)$ field elements and $(10m + O(\sqrt{m} + n))$ $\mathbb{F}$. The verifier cost is $O(\log m)$ $\mathbb{P} + O(\log m \log\log m)$ $\mathbb{F}$, and the proof size is $O(\log m \log\log m)$.

The missing piece in the Celer lookup argument is to compile the PIOP using a multilinear PCS. The main bottleneck in this compilation is to efficiently prove the final random evaluation $\tilde{a}(r')$. To this end, we need a multilinear PCS whose opening group cost is sublinear in m, while maintaining proof size and verifier time polylogarithmic in m. Fortunately, Dory PCS [Lee21] precisely provides these properties.

We formally analyze the completeness and soundness of Celer in Section 4.5.

4 Methodology

In this section, we present the concrete construction of Celer. The starting point of Celer builds on an observation from LogUp [Hab22,EKRN24]: the lookup relation $a \subseteq T$ is equivalent to the existence of integers $e_1, \cdots, e_n \in [0, m]$ such that the polynomials $h(X) = \prod_{i=1}^{m}(X - a_i)$ and $g(X) = \prod_{i=1}^{n}(X - T_i)^{e_i}$ are identical. The integers $e_1, \cdots, e_n$ represent the multiplicities with which each table element $T_1, \cdots, T_n$ appears in the query vector a. The prover proves the polynomial identity by evaluating both $g(X)$ and $h(X)$ at a random point r chosen by the verifier, and showing that $g(r) = h(r)$ holds. In the following sections, we will introduce how to efficiently prove $g(r)$ and $h(r)$, respectively.

4.1 Efficiently proving $g(r)$

We first design an efficient method to prove $g(r) = \prod_{i=1}^{n}(r - T_i)^{e_i}$, which takes the form of a product of powers. The Pippenger algorithm [Pip76] is a classical technique for efficiently computing Multi-Scalar Multiplication (MSM). As observed in [Ber02], the Pippenger algorithm can also be adapted to efficiently compute products of powers. Inspired by this, we design a specialized protocol to prove $g(r)$ based on proving the execution of the Pippenger algorithm.

Assume each exponent e_i is decomposed into w digits as $e_i = \sum_{j=1}^{w} s^{j-1} \cdot E_{j,i}$, under base $s = \lceil (m+1)^{\frac{1}{w}} \rceil$. The procedure of using the Pippenger algorithm to compute $g(r)$ could be summarized as:
(1) Compute each bucket value $C_{j,v}$ using $r - T_i$ and decomposed digits E_j: $C_{j,v} = \prod_{E_{j,i}=v}(r - T_i)$.
(2) Compute each suffix product $S_{j,v}$ based on the bucket values C_j: $S_{j,v} =$

$\prod_{\ell=v}^{s-1} C_{j,\ell}$.
(3) Compute R_j based on the suffix product values S_j: $R_j = \prod_{v=1}^{s-1} S_{j,v}$; then aggregate R_j together to obtain $g(r)$: $g(r) = \prod_{j=1}^{w} R_j^{s^{j-1}}$.

The concrete procedure of computing $g(r)$ using the Pippenger algorithm is illustrated in Algorithm 1. A naive approach for proving $g(r)$ is to commit to vector e and prove the execution of Algorithm 1 by treating it as a general program. It is easy to analyze that Algorithm 1 runs in $O(n+m^{\frac{1}{w}})$ time. Leveraging existing proof techniques for memory read-write operations [FKL$^+$21,YH24,Set20], we can prove Algorithm 1 in $O(n+m^{\frac{1}{w}})$ time. However, since existing techniques for proving memory read-write operations incur substantial constant factors, this approach for proving $g(r)$ is still costly.

To address this issue, we design a more efficient way to prove the correctness of $g(r)$. The key idea of our method is to avoid explicitly committing E and maintaining each bucket value $C_{j,v}$. Updating the bucket values C_j requires frequent memory read and write operations. Such operations are inherently ZK-unfriendly, as existing ZKP constructions for random access memory(RAM) [FKL$^+$21,YH24,Set20,ZGK$^+$18] incur significant overhead when verifying dynamic memory accesses. To bypass proving C_j, we encode the information of digit vector E_j into vector U_j and p_j, and prove the correctness of $S_{j,v}$ based on U_j and p_j.

To elaborate, we encode the information of digit vector E_j into vector U_j and p_j as follows: For each bucket value $C_{j,v}$, we enumerate over all of the T_i's contributing to this bucket value (all T_i such that $E_{j,i} = v$), then we concatenate all of these values into a sequence U_j by the order of v. We use p_j to mark the segments formed by different buckets, $p_{j,v}$ is the smallest element index in U_j corresponding to $E_{j,i} \geq v$. Formally, we have:

$$U_j = \bigoplus_{v=0}^{s-1} [T_i | E_{j,i} = v]$$
$$p_{j,v} = |\{i | E_{j,i} < v\}| + 1$$

where $\oplus$ denotes the concatenation of vectors. An example of encoding the information of E_j into U_j and p_j could be found in Figure 1.

Our central observation is that, each $S_{j,v}$ is one of the suffix product of U_j, and thus can be easily obtained from U_j and p_j. Based on the construction of U_j and p_j, for $\forall v$, $\{U_{j,\ell} | \ell \geq p_{j,v}\} = \{T_i | E_{j,i} \geq v\}$. Thus, we have:

$$S_{j,v} = \prod_{E_{j,i} \geq v} (r - T_i) = \prod_{\ell=p_{j,v}}^{n} (r - U_{j,\ell})$$

We define the suffix product vector of $r - U_j$ as D_j, and $D_{j,i} = \prod_{\ell=i}^{n} (r - U_{j,\ell})$. Based on the above equation, we observe the following identity connecting D_j and S_j:

$$D_{j,p_{j,v}} = \prod_{\ell=p_{j,v}}^{n} (r - U_{j,\ell}) = S_{j,v} \tag{6}$$

To give a more illustrative explanation of the above equation, we revisit our example presented in Figure 1. Consider $S_{j,2} = \prod_{E_{j,i} \geq 2}(r - T_i) = (r - T_6)(r - T_1)(r - T_2)(r - T_4)$ as an example. Since $p_{j,2} = 3$, we have $D_{j,p_{j,v}} = D_{j,3} = \prod_{\ell=3}^{6}(r - U_{j,\ell}) = (r - U_{j,3}) \cdot (r - U_{j,4}) \cdot (r - U_{j,5}) \cdot (r - U_{j,6}) = (r - T_6) \cdot (r - T_1) \cdot (r - T_2) \cdot (r - T_4)$, which matches $S_{j,2}$ as expected.

Equation 6 thus enables a direct connection between S_j and the pair (U_j, p_j), allowing us to use the latter as a ZKP-friendly encoding of E_j in the proof of $g(r)$. Specifically, the prover proves $g(r)$ by proving the correctness of each value of R_j. Since $R_j = \prod_{v=1}^{s-1} S_{j,v}$, it reduces to proving the value of each $S_{j,v}$ is correct. And leveraging U_j and p_j as witness, the correctness of $S_{j,v}$ could be proven based on Equation 6.

In the protocol, instead of committing to E_j, the prover commits to U_j and p_j before $\mathcal{V}$ samples the challenge r. Then, $\mathcal{P}$ proves to $\mathcal{V}$ that U_j is a permutation of T. Upon receiving the challenge r, $\mathcal{P}$ sends the claims on R_j to $\mathcal{V}$, and also sends the commitments to D_j, for $j = 1, \cdots, w$. $\mathcal{V}$ checks that the previous claim on $g(r)$ equals $\prod_{j=1}^{w} R_j^{s^{j-1}}$. To prove the correctness of each R_j, $\mathcal{P}$ reduces the task to proving three relations:

(1) $D_{j,i} = \prod_{\ell=i}^{n}(r - U_{j,\ell})$ for all $j = 1, \cdots, w$, $i = 1, \cdots, n$

(2) $S_{j,v} = D_{j,p_{j,v}}$ for all $j = 1, \cdots, w$, $v = 1, \cdots, s-1$

(3) $R_j = \prod_{v=1}^{s-1} S_{j,v}$ for all $j = 1, \cdots, w$.

A naive solution for proving Relation (1) is to transform it into an arithmetic circuit of constant depth, taking U_j and D_j as inputs, and then apply the GKR protocol. This gives prover cost $O(n)$ $\mathbb{F}$. However, the resulting verifier is not succinct unless the wiring can be evaluated efficiently.

We avoid this issue by shifting only the U_j vector. Relation (1) is equivalent to $D_{j,i} = D_{j,i+1}(r - U_{j,i}), i = 1, \ldots, n$. Let U'_j be the shifted version of U_j, where

$$U'_{j,1} = \perp, \qquad U'_{j,i+1} = U_{j,i}, \quad i = 1, \ldots, n.$$

Here $\perp$ is a dummy value. The above relation becomes $D_{j,i} = D_{j,i+1}(r - U'_{j,i+1}), i = 1, \ldots, n$.

The verifier samples a random challenge $\alpha \in \mathbb{F}$ and the prover proves their random linear combination: $\sum_{i=1}^{n} \alpha^i D_{j,i} = \sum_{i=1}^{n} \alpha^i D_{j,i+1}(r - U'_{j,i+1})$.

By re-indexing the right-hand side with $t = i + 1$, this becomes

$$\begin{aligned} \sum_{i=1}^{n} A_i D_{j,i} &= \sum_{t=2}^{n+1} \alpha^{t-1} D_{j,t}(r - U'_{j,t}) \\ &= \sum_{t=1}^{2n} B_t D_{j,t}(r - U'_{j,t}) \end{aligned}$$

where $A_i = \alpha^i, B_t = \mathbf{1}_{2\le t\le n+1}\alpha^{t-1}$. The LHS of the above equation could be proven by the sumcheck protocol; it is apparent that the verifier can efficiently evaluate $\tilde{A}$ at a random point [BCHO22,QGYZ25].

To prove the RHS, the prover runs a sumcheck over a Boolean domain of size $N = 2n = 2^\ell$: $\sum_{x\in\{0,1\}^\ell} \widetilde{B}(x)\widetilde{D}_j(x)\big(r-\widetilde{U}'_j(x)\big)$. The verifier needs to evaluate the public oracle $\widetilde{B}$ succinctly. First consider the vector $Q = (1, \alpha, \alpha^2, \ldots, \alpha^{n-1}, 0, \ldots, 0)$. Since $N = 2n$, the nonzero part of Q occupies exactly the first half of the domain. Therefore, its multilinear extension has a tensor-product form [BCHO22].

The vector B is obtained from Q by setting the first entry to 0 and adding the missing value α^n at index $n+1$. Let $\mathbf{e}_i$ denote the one-hot vector that is 1 at index i and 0 elsewhere, and let $\widetilde{\mathbf{e}}_i$ be its multilinear extension. Then $B = Q - \mathbf{e}_1 + \alpha^n\mathbf{e}_{n+1}$. Therefore, for an evaluation point τ,

$$\widetilde{B}(\tau) = \widetilde{Q}(\tau) - \widetilde{\mathbf{e}}_1(\tau) + \alpha^n\widetilde{\mathbf{e}}_{n+1}(\tau).$$

Both one-hot evaluations are efficient. Thus $\widetilde{B}(\tau)$ can be evaluated by the verifier in $O(\log n)$ field operations.

Relation (2) can be efficiently proven using the indexed lookup technique [STW24]. More precisely, relation (2) can be transformed into the following set inclusion relationship:

$$\{(p_{j,1}, S_{j,1}), \cdots, (p_{j,s-1}, S_{j,s-1})\} \subseteq \{(1, D_{j,1}), \cdots, (n, D_{j,n})\}$$

This set inclusion relationship can be proven using the LogUp protocol. To begin, $\mathcal{V}$ sends a random challenge $\gamma \in \mathbb{F}$. Let c_k be the multiplicities of $(k, D_{j,k})$ in $\{(p_{j,1}, S_{j,1}), \cdots, (p_{j,s-1}, S_{j,s-1})\}$, $\mathcal{P}$ proves the inclusion by showing the following identity holds:

$$\sum_{i=1}^{s-1} \frac{1}{p_{j,i} + \gamma \cdot S_{j,i}} = \sum_{k=1}^{n} \frac{c_k}{k + \gamma \cdot D_{j,k}}$$

In the LogUp protocol, $\mathcal{P}$ commits to two auxiliary vectors f_j and f'_j, where $f_{j,i} = \frac{1}{p_{j,i}+\gamma\cdot S_{j,i}}$, $f'_{j,k} = \frac{c_k}{k+\gamma\cdot D_{j,k}}$, then proves the following equations with the sumcheck protocol:

$$\begin{aligned} \sum_{i=1}^{s-1} f_{j,i} &= \sum_{k=1}^{n} f'_{j,k} \\ f_{j,i} \cdot (p_{j,i} + \gamma \cdot S_{j,i}) &= 1 \\ f'_{j,k} \cdot (k + \gamma \cdot D_{j,k}) - c_k &= 0 \end{aligned}$$

Proving relation (2) results in prover work of committing to $O(n + m^{\frac{1}{w}})$ field elements and $O(n + m^{\frac{1}{w}})$ $\mathbb{F}$.

Relation (3) has the same form as the grand product check and can be handled using existing techniques from Quark [SL20].

Hence, we complete the proof of $g(r)$ with prover work of committing to $O(n + m^{\frac{1}{w}})$ field elements and $O(n + m^{\frac{1}{w}})$ $\mathbb{F}$.

Protocol 1 (Celer PIOP) • *Input: Oracle access to* $\tilde{a} : \mathbb{F}^{\log m} \to \mathbb{F}$*, the multilinear extension of query vector* a*; Denote bucket number* $s = \lceil (m+1)^{\frac{1}{w}} \rceil$*.*
• $\mathcal{P}$*: Compute* $e_1, \cdots, e_n$ *as the counts of* $T_1, \cdots, T_n$ *in the query vector* a*, and decompose each* e_i *into* $E_{j,i}$ *as:* $e_i = \sum_{j=1}^{w} s^{j-1} \cdot E_{j,i}$*. Then,* $\mathcal{P}$ *constructs the vectors* $U_1, \cdots, U_w$ *based on* $U_j = \bigoplus_{v=0}^{s-1}[T_i | E_{j,i} = v]$*, and computes* $p_1, \cdots, p_w$ *accordingly.*
• $\mathcal{P} \to \mathcal{V}$*: 2*$w$ *different multilinear polynomial commitments to* $\tilde{U_1}, \cdots, \tilde{U_w}$*,* $\tilde{p_1}, \cdots, \tilde{p_w}$*.*
• $\mathcal{V} \to \mathcal{P}$*: Random challenge* $r \in F$*.*
• $\mathcal{P} \to \mathcal{V}$*: Claim* $h(r) \in \mathbb{F}$*,* $R_1, \cdots, R_w \in \mathbb{F}$*,* w *different multilinear polynomial commitments to* $\tilde{D_1}, \cdots, \tilde{D_w}$*,* $\tilde{S_1}, \cdots, \tilde{S_w}$*.*
• $\mathcal{V}$*: Check* $h(r) \stackrel{?}{=} \prod_{j=1}^{w} R_j^{s^{j-1}}$*.*
• $\mathcal{P} \leftrightarrow \mathcal{V}$*: Run grand product check protocol to check* $h(r) \stackrel{?}{=} \prod_{i=1}^{m}(r - a_i)$*. The protocol reduces to prover's claim on* $\tilde{a}(r')$*, which is verified by* $\mathcal{V}$*'s oracle access to* $\tilde{a}$*.*
• $\mathcal{P} \leftrightarrow \mathcal{V}$*: For* $j = 1, \cdots, w$*, run grand product check protocol to check* $R_j \stackrel{?}{=} \prod_{v=1}^{s-1} S_{j,v}$*.*
• $\mathcal{P} \leftrightarrow \mathcal{V}$*: For* $j = 1, \cdots, w$*, run sumcheck protocol to prove that* $D_{j,i} \stackrel{?}{=} \prod_{\ell=i}^{n}(r - U_{j,\ell}), i = 1, \cdots, n$*.*
• $\mathcal{P} \leftrightarrow \mathcal{V}$*: For* $j = 1, \cdots, w$*, run the LogUp protocol to check* $\{(p_{j,1}, S_{j,1}), \cdots, (p_{j,s-1}, S_{j,s-1})\} \subseteq \{(1, D_{j,1}), \cdots, (n, D_{j,n})\}$*.*
• $\mathcal{P} \leftrightarrow \mathcal{V}$*: For* $j = 1, \cdots, w$*, run grand product check protocol to check* U_j *is a permutation of* $\{T_1, \cdots, T_n\}$*.*

Fig. 2: Celer PIOP.

4.2 Proving $h(r)$

Next, the prover needs to prove the claim on $h(r) = \prod_{i=1}^{m}(X - a_i)$. We observe that $h(r)$ could be proven using the grand product check protocol from Quark [SL20] (introduced in Section 2.3). It reduces the claim on $h(r)$ to a claim on $\tilde{a}(r')$ for some $r' \in \mathbb{F}^{\lceil \log m \rceil}$. The grand product check protocol incurs a prover cost of committing to $\frac{m}{\log^3 m}$ field elements and $O(m)$ $\mathbb{F}$. Combining this with the construction for proving $g(r)$ in Section 4.1, we obtain a lookup PIOP with prover commitment sublinear in query size m.

The missing piece of Celer lookup argument is to use a multilinear PCS to compile the PIOP. The challenge here is that the prover has to efficiently open the committed vector a at a random point r' (opening group cost is sublinear in m), while maintaining proof size and verifier time polylogarithmic in m. Fortunately, Dory PCS [Lee21] precisely provides these properties.

4.3 Putting everything together

In Celer lookup PIOP, $\mathcal{P}$ first computes $e_1, \cdots, e_n$ as the counts of $T_1, \cdots, T_n$ in query a. These counts are then decomposed into digit vectors $E_1, \ldots, E_w$ (in

our implementation, we set the digit decomposition parameter as $w = 2$). Based on T and the decomposed digit vector E_j, $\mathcal{P}$ constructs the auxiliary vectors U_j and p_j for each $j = 1, \ldots, w$, and commits to them.

Subsequently, $\mathcal{V}$ sends a random challenge $r \in \mathbb{F}$ to $\mathcal{P}$. Upon receiving r, $\mathcal{P}$ sends the claim $h(r) = g(r)$, along with the values $R_1, \ldots, R_w$ and the commitments to S_j and D_j for each $j = 1, \ldots, w$. $\mathcal{V}$ then checks that the claim on $g(r)$ is consistent with the claimed R_j values via the relation $g(r) = \prod_{j=1}^{w} R_j^{s^{j-1}}$.

To prove the claim on $h(r)$, $\mathcal{P}$ engages in a grand product check with $\mathcal{V}$, which reduces the claim on $h(r)$ to a claim on the committed query vector a. This is followed by an opening of the commitment to a using the Dory PCS, thereby completing the verification of $h(r)$.

Finally, for each $j = 1, \ldots, w$, $\mathcal{P}$ proves the following relations: (1) That $D_{j,i} \stackrel{?}{=} \prod_{\ell=i}^{n}(r - U_{j,\ell}), i = 1, \cdots, n$, using the sumcheck protocol (2) That $R_j = \prod_{v=1}^{s-1} S_{j,v}$ using a grand product check. (3) That $D_{j,p_{j,v}} = S_{j,v}$, using the indexed lookup technique together with the LogUp protocol. (4) That each U_j is a permutation of the table vector T, using the grand product check protocol [SL20].

For clarity, the whole procedure of Celer lookup PIOP is illustrated in Protocol 1.

4.4 Cost Analysis

The following theorem summarizes the performance results achieved by Celer.

Theorem 1. *The properties of Celer lookup PIOP are as follows:*

- *prover work: committing to* $\frac{m}{\log^3 m} + O(m^{\frac{1}{w}} + n)$ *field elements, and performing* $(10m + O(m^{\frac{1}{w}} + n))$ $\mathbb{F}$
- *proof size:* $O(\log m \log\log m)$
- *verifier time:* $O(\log m \log\log m)$

Note that table size n does not appear in the complexity of proof size and verifier time because, the n-related terms appear with the same asymptotic order as m, and are thus dominated under the $m \gg n$ assumption of our paper.

Prover Time Analysis. We first analyze the prover cost in Celer PIOP, which can be divided into the cost of commitments and the cost of field operations. On the commitment side, for proving each R_j, the prover commits to U_j, p_j, S_j, D_j, with $O(n + s)$ values in total. In addition, proving the grand product check for $h(r)$ requires committing to $\frac{m}{\log^3 m}$ values. Putting these together, in Celer PIOP the prover commits to $\frac{m}{\log^3 m} + O(\sqrt{m} + n)$ field elements overall.

Turning to field operations, the grand product check for evaluating $h(r)$ costs $10m$ $\mathbb{F}$[5]. In addition, the grand product check for U_j requires $O(n)$ $\mathbb{F}$, and proving

[5] Based on the analysis in [BDT24], a sumcheck protocol with degree = 3 on a $2^{\log m}$-size hypercube consumes $10m$ field operations. A grand product check on the same size of input performs sumcheck on size $2^{\log m - 1}, 2^{\log m - 2}, \cdots$, and thus also costs $10m$ field operations.

correctness for each R_j further costs $O(s)$ $\mathbb{F}$. Finally, opening U_j, p_j, S_j, D_j adds another $O(n+s)$ $\mathbb{F}$. Summing these contributions, the total number of field operations in Celer PIOP is $10m + O(m^{\frac{1}{w}} + n)$ $\mathbb{F}$.

In conclusion, the prover work of Celer PIOP is committing to $\frac{m}{\log^3 m} + O(m^{\frac{1}{w}} + n)$ field elements and $(10m + O(m^{\frac{1}{w}} + n))$ $\mathbb{F}$.

Compared with the most efficient baseline, LogUp-GKR, Celer achieves roughly a $4\times$ reduction in field-operation costs. This improvement stems from the smaller constants of the grand-product check relative to the GKR protocol.

Compared with Lasso, the advantage of Celer comes from two sources. First, Lasso commits to $3m$ field elements for memory checking, whereas Celer commits to only a sublinear number of elements in (m). Second, Lasso invokes the grand-product check twice for memory checking, while Celer invokes it only once, resulting in a smaller constant factor in field operations.

Proof Size Analysis. The proof size of Celer lookup PIOP is dominated by the grand product check in proving $h(r)$, sending $O(\log m \log\log m)$ field elements.

Verifier Time Analysis. Celer lookup PIOP's verifier work consists of (1) the verifier time of the grand product check in proving $h(r)$. (2) the verifier time for proving $g(r)$. Part (1) incurs $O(\log m \log\log m)$, part (2) incurs $O(\log n \log\log n)$. Thus, the total verifier cost is $O(\log m \log\log m)$.

4.5 Security analysis

The following theorem summarizes the security properties of Celer.

Theorem 2. *Celer achieves completeness error $O(\frac{m^{\frac{1}{w}}+n}{|\mathbb{F}|})$ and soundness error $O(\frac{m\cdot n}{|\mathbb{F}|})$.*

Proof Sketch. The soundness of Celer hinges on ensuring that each R_j is computed as the product of $s-1$ entries from the suffix product vector S_j, which bounds the degree of $g(r)$ by $O(m \cdot n)$. Since U_j and p_j are committed before the challenge r is sampled, the polynomial $g(\cdot)$ is fixed in advance. Soundness then follows from the Schwartz-Zippel lemma [Sch80]. Notably, although Celer does not enforce the well-formedness of p_j, the resulting soundness error remains negligible.

Proof. The completeness error of Celer arises from the invocation of the LogUp protocol, which is parameterized by query size $O(m^{1/w})$ and table size $O(n)$. According to the completeness guarantee of LogUp [Hab22], the completeness error of Celer is bounded by $O(\frac{m^{\frac{1}{w}}+n}{|\mathbb{F}|})$.

We now derive the soundness error. To simplify the analysis, we first consider the case where all of the $\mathcal{V}$'s checks on $g(r)$ pass. By the soundness of grand product check on R_j, if $\mathcal{V}$'s check on $R_j \stackrel{?}{=} \prod_{v=1}^{s-1} S_{j,v}$ passes, $\mathcal{P}$'s claim on $h(r) = \prod_{j=1}^{w}(\prod_{v=1}^{s-1} S_{j,v})^{s^{j-1}}$ holds with at least $1 - O(\frac{\log m \log\log m}{|\mathbb{F}|})$.

In addition, the LogUp protocol in proving $g(r)$ ensures that if $\mathcal{V}$'s checks pass, then $\{(p_{j,1}, S_{j,1}), \cdots, (p_{j,s-1}, S_{j,s-1})\} \subseteq \{(1, D_{j,1}), \cdots, (n, D_{j,n})\}$ holds

with probability at least $1-O(\frac{m^{\frac{1}{w}}+n}{|\mathbb{F}|})$. This implies that, with high probability, $D_{j,p_{j,v}} = S_{j,v}$ for all v.

Consequently, with high probability, we have $g(r) = \prod_{j=1}^{w}(\prod_{v=1}^{s-1} S_{j,v})^{s^{j-1}} = \prod_{j=1}^{w}(\prod_{v=1}^{s-1} D_{j,p_{j,v}})^{s^{j-1}}$.

By the soundness of the sumcheck protocol, if $\mathcal{V}$'s checks on the claimed values of $D_{j,i}$ pass, then each $D_{j,i} = \prod_{\ell=i}^{n}(r - U_{j,\ell})$ holds with probability at least $1 - O(\frac{\log n}{|\mathbb{F}|})$.

Applying the union bound, the following equation holds with high probability:

$$g(r) = \prod_{j=1}^{w}(\prod_{v=1}^{s-1}(\prod_{\ell \geq p_{j,v}}(r - U_{j,\ell})))^{s^{j-1}} \tag{7}$$

Denote $E'_{j,\ell} = \sum_{v=1}^{s-1} \mathbb{1}(\ell \geq p_{j,v})$, where $\mathbb{1}(\cdot)$ is the indicator function. We can rewrite Equation 7 as:

$$g(r) = \prod_{j=1}^{w}(\prod_{\ell=1}^{n}(r - U_{j,\ell})^{E'_{j,\ell}})^{s^{j-1}} \tag{8}$$

Recall that U_j is a permutation of T (enforced by a grand product check). If $\mathcal{V}$'s check passes, then with high probability, for each j there exists a mapping $\pi_j(i)$ such that $T_i = U_{j,\pi_j(i)}$. Then:

$$\prod_{\ell=1}^{n}(r - U_{j,\ell})^{E'_{j,\ell}} = \prod_{i=1}^{n}(r - T_i)^{E'_{j,\pi_j(i)}}$$

Plugging the above equation into Equation 8, we have:

$$g(r) = \prod_{i=1}^{n}(r - T_i)^{\sum_{j=1}^{w} E'_{j,\pi_j(i)} \cdot s^{j-1}} = \prod_{i=1}^{n}(r - T_i)^{e'_i}$$

where $e'_i = \sum_{j=1}^{w} E'_{j,\pi_j(i)} \cdot s^{j-1}$.

Now we conclude that, if all $\mathcal{V}$'s check for $g(r)$ passes, then $g(r) = \prod_{i=1}^{n}(r - T_i)^{e'_i}$ holds with probability at least $1 - O(\frac{m^{\frac{1}{w}}+n}{|\mathbb{F}|})$. Note that $E'_{j,\pi_j(i)} \leq s-1$, since $e'_i = \sum_{j=1}^{w} E'_{j,\pi_j(i)} \cdot s^{j-1}$, we have $e'_i \leq s^w - 1$. Due to $s = \lceil (m+1)^{\frac{1}{w}} \rceil$, $e'_i \leq m + \Theta\left(m^{1-1/w}\right)$. Therefore, the degree of polynomial $g(X) = \prod_{i=1}^{n}(X - T_i)^{e'_i}$ is at most $O(m \cdot n)$.

On the other hand, if $\mathcal{V}$'s check on $h(r)$ passes, then by the soundness of the grand product check, $h(r) = \prod_{i=1}^{m}(r - a_i)$ holds with probability of at least $1 - O(\frac{\log m \log\log m}{|\mathbb{F}|})$.

Note that both e' and a are fixed by their commitment before the verifier samples r. So if $a \not\subseteq T$, the two polynomials $\prod_{i=1}^{m}(X - a_i)$ and $\prod_{i=1}^{n}(X - T_i)^{e'_i}$

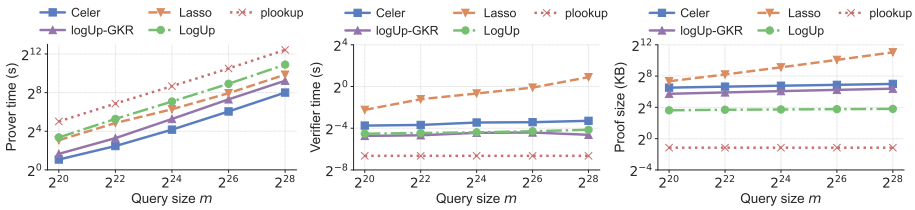
are distinct polynomials, and by the Schwartz–Zippel lemma [Sch80], they can agree at most with probability $O(\frac{m \cdot n}{|\mathbb{F}|})$. In conclusion, the soundness error of Celer is at most $O(\frac{m \cdot n}{|\mathbb{F}|})$.

5 Experiments

In this section, we present a comprehensive evaluation of our efficient lookup scheme Celer.

5.1 Implementations

Software. Our implementation uses the BN254 elliptic curve, which provides 128-bit security. We develop our lookup protocols in C++ language using the mcl library.
Hardware. All the experiments are conducted on a server with 256 GB memory and Intel(R) Xeon(R) Platinum 8460Y+ 64 CPU cores. We always use a single thread to execute the experiments.
Baselines. In this paper, we compare the performance of our lookup argument against several widely adopted lookup arguments, including plookup [GW20], LogUp [Hab22], logUp-GKR [PH23], and Lasso [STW24] (EUROCRYPT 2024). For Lasso, we adopt its official implementation [Las24]. For plookup, LogUp, and logUp-GKR, we implement them ourselves.



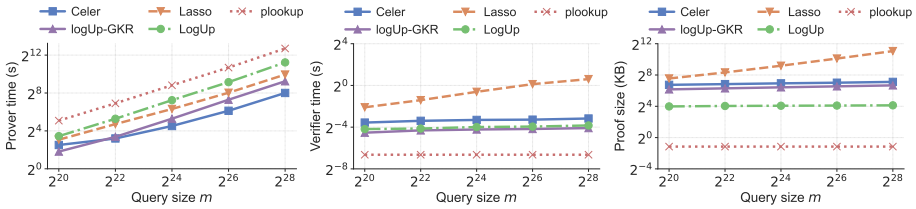
(a) (b) (c)

Fig. 3: Results for table size $n = 2^8$.

5.2 Performance of Celer

We first compare Celer lookup argument and the baselines under different table sizes $n = 2^8, 2^{16}$ and query sizes $m = 2^{20}, 2^{22}, 2^{24}, 2^{26}, 2^{28}$, to test the performance of our method under the scenario of $m \gg n$. The results are shown in Figure 3 and Figure 4.

As shown in Figure 3a and Figure 4a, Celer consistently demonstrates significantly better prover performance compared to the baseline methods. For



(a) (b) (c)

Fig. 4: Results for table size $n = 2^{16}$.

example, under $m = 2^{28}$ and $n = 2^{16}$, Celer achieves a prover time of only 257 seconds, while logUp-GKR requires 604 seconds, Lasso requires 986 seconds, LogUp requires 2402 seconds, plookup requires 6655 seconds. This means that Celer outperforms all of the baselines by $2.4 \sim 25.9\times$ on prover time. This is mainly due to the highly efficient design of Celer lookup protocol.

We can also observe that, in most cases, the prover time of Celer grows almost linearly with m. This is because Celer lookup argument is bottlenecked by the grand product check protocol plus the opening of Dory PCS, costing $O(m)$ $\mathbb{F}$.

The verifier time of Celer and baseline methods is shown in Figures 3b and 4b. Across all settings, Celer maintains a verifier time of ≤ 0.11 seconds, demonstrating its low verification cost. Compared to LogUp, Celer incurs an additional $O(\log m \log\log m)$ $\mathbb{F}$. Nonetheless, its verifier time remains within the same order of magnitude. This is primarily because both schemes incur pairing operations from PCS openings, which dominate the overall verifier cost due to their high expense.

Figures 3c and 4c report the proof sizes of Celer and baseline schemes. Celer consistently produces proofs of size at most 248 KB across different settings, making it well-suited for practical deployment. Although logUp-GKR has an asymptotic proof size of $O(\log^2 m)$, its concrete proofs are slightly smaller than those of Celer. This is because the $O(\log^2 m)$ proof components in logUp-GKR are field elements, each of which is substantially smaller than the $\mathbb{G}_T$ elements used in Dory evaluation proofs. Moreover, Celer commits to and evaluates a bunch of polynomials, further increasing its concrete proof size. For scenarios that are extremely sensitive to proof size, proof recursion [BGH19] can be employed to further reduce the proof size to $O(\log m)$.

Lastly, in our experiments, the verifier time and proof size of Lasso grow rapidly as m increases. This is due to its use of the Hyrax PCS in the official implementation [Zhu24], which incurs $O(\sqrt{m})$ overhead in both metrics.

6 Acknowledgments

We thank Ariel Gabizon for helpful discussions and for drawing our attention to logUp-GKR as an important point of comparison.

References

[AST24] Arasu Arun, Srinath Setty, and Justin Thaler. Jolt: Snarks for virtual machines via lookups. In *Advances in Cryptology – EUROCRYPT 2024: 43rd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zurich, Switzerland, May 26–30, 2024, Proceedings, Part VI*, 2024.

[BBB+18] Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell. Bulletproofs: Short proofs for confidential transactions and more. In *2018 IEEE symposium on security and privacy (SP)*, pages 315–334. IEEE, 2018.

[BCG+18] Jonathan Bootle, Andrea Cerulli, Jens Groth, Sune Jakobsen, and Mary Maller. Arya: Nearly linear-time zero-knowledge proofs for correct program execution. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 595–626. Springer, 2018.

[BCHO22] Jonathan Bootle, Alessandro Chiesa, Yuncong Hu, and Michele Orru. Gemini: Elastic snarks for diverse environments. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 427–457. Springer, 2022.

[BDT24] Suyash Bagad, Yuval Domb, and Justin Thaler. The sum-check protocol over fields of small characteristic. *Cryptology ePrint Archive*, 2024.

[Ber02] Daniel J Bernstein. Pippenger's exponentiation algorithm. *Preprint.*, 2002. URL: `http://cr.yp.to/papers/pippenger.pdf`.

[BFS20] Benedikt Bünz, Ben Fisch, and Alan Szepieniec. Transparent snarks from dark compilers. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 677–706. Springer, 2020.

[BGH19] Sean Bowe, Jack Grigg, and Daira Hopwood. Recursive proof composition without a trusted setup. *Cryptology ePrint Archive*, 2019.

[BGL+23] James Bell, Adrià Gascón, Tancrède Lepoint, Baiyu Li, Sarah Meiklejohn, Mariana Raykova, and Cathie Yun. {ACORN}: input validation for secure aggregation. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 4805–4822, 2023.

[CBBZ23] Binyi Chen, Benedikt Bünz, Dan Boneh, and Zhenfei Zhang. Hyperplonk: Plonk with linear-time prover and high-degree custom gates. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 499–530. Springer, 2023.

[CFF+24] Matteo Campanelli, Antonio Faonio, Dario Fiore, Tianyu Li, and Helger Lipmaa. Lookup arguments: improvements, extensions and applications to zero-knowledge decision trees. In *IACR International Conference on Public-Key Cryptography*, pages 337–369. Springer, 2024.

[CMT12] Graham Cormode, Michael Mitzenmacher, and Justin Thaler. Practical verified computation with streaming interactive proofs. In *Proceedings of the 3rd Innovations in Theoretical Computer Science Conference*, pages 90–112, 2012.

[DCB24] Trisha Datta, Binyi Chen, and Dan Boneh. Veritas: Verifying image transformations at scale. *Cryptology ePrint Archive*, 2024.

[Dor24] Daniel Dore. Tassle: Lasso for the commitment-phobic. *Cryptology ePrint Archive*, 2024.

[EFG22] Liam Eagen, Dario Fiore, and Ariel Gabizon. cq: Cached quotients for fast lookups. *Cryptology ePrint Archive*, 2022.

[EKRN24] Liam Eagen, Sanket Kanjalkar, Tim Ruffing, and Jonas Nick. Bulletproofs++: next generation confidential transactions via reciprocal set membership arguments. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 249–279. Springer, 2024.

[EZC+25] Jens Ernstberger, Chengru Zhang, Luca Ciprian, Philipp Jovanovic, and Sebastian Steinhorst. Zero-Knowledge Location Privacy via Accurate Floating-Point SNARKs . In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 57–57, 2025.

[FKL+21] Nicholas Franzese, Jonathan Katz, Steve Lu, Rafail Ostrovsky, Xiao Wang, and Chenkai Weng. Constant-overhead zero-knowledge for ram programs. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, pages 178–191, 2021.

[FS86] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In *Conference on the theory and application of cryptographic techniques*, pages 186–194. Springer, 1986.

[GKR15] Shafi Goldwasser, Yael Tauman Kalai, and Guy N Rothblum. Delegating computation: interactive proofs for muggles. *Journal of the ACM (JACM)*, 62(4):1–64, 2015.

[GLH+25] Yanpei Guo, Xuanming Liu, Kexi Huang, Wenjie Qu, Tianyang Tao, and Jiaheng Zhang. {DeepFold}: Efficient multilinear polynomial commitment from {Reed-Solomon} code and its application to zero-knowledge proofs. In *34th USENIX Security Symposium (USENIX Security 25)*, pages 3497–3516, 2025.

[GQW+26] Yanpei Guo, Wenjie Qu, Linyu Wu, Shengfang Zhai, Lionel Z Wang, Ming Xu, Yue Liu, Binhang Yuan, Dawn Song, and Jiaheng Zhang. Immaculate: A practical llm auditing framework via verifiable computation. *arXiv preprint arXiv:2602.22700*, 2026.

[GW20] Ariel Gabizon and Zachary J Williamson. plookup: A simplified polynomial protocol for lookup tables. *Cryptology ePrint Archive*, 2020.

[GWC19] Ariel Gabizon, Zachary J Williamson, and Oana Ciobotaru. Plonk: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge. *Cryptology ePrint Archive*, 2019.

[Hab22] Ulrich Haböck. Multivariate lookups based on logarithmic derivatives. *Cryptology ePrint Archive*, 2022.

[HACF25] Hossein Hafezi, Gaspard Anthoine, Matteo Campanelli, and Dario Fiore. Sok: Lookup table arguments. *Cryptology ePrint Archive*, 2025.

[HCL+24] Meng Hao, Hanxiao Chen, Hongwei Li, Chenkai Weng, Yuan Zhang, Haomiao Yang, and Tianwei Zhang. Scalable zero-knowledge proofs for nonlinear functions in machine learning. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 3819–3836, 2024.

[HG16] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (gelus). *arXiv preprint arXiv:1606.08415*, 2016.

[KHSS22] Daniel Kang, Tatsunori Hashimoto, Ion Stoica, and Yi Sun. Scaling up trustless dnn inference with zero-knowledge proofs. *arXiv preprint arXiv:2210.08674*, 2022.

[KZG10] Aniket Kate, Gregory M Zaverucha, and Ian Goldberg. Constant-size commitments to polynomials and their applications. In *Advances in Cryptology-ASIACRYPT 2010: 16th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 5-9, 2010. Proceedings 16*, pages 177–194. Springer, 2010.

[Las24] Lasso. Official implementation of lasso., 2024. URL: `https://github.com/a16z/jolt/tree/lasso`.

[LBV+23] Hidde Lycklama, Lukas Burkhalter, Alexander Viand, Nicolas Küchler, and Anwar Hithnawi. Rofl: Robustness of secure federated learning. In *2023 IEEE Symposium on Security and Privacy (SP)*, pages 453–476. IEEE, 2023.

[Lee21] Jonathan Lee. Dory: Efficient, transparent arguments for generalised inner products and polynomial commitments. In *Theory of cryptography conference*, pages 1–34. Springer, 2021.

[LWQ+24] Tao Lu, Haoyu Wang, Wenjie Qu, Zonghui Wang, Jinye He, Tianyang Tao, Wenzhi Chen, and Jiaheng Zhang. An efficient and extensible zero-knowledge proof framework for neural networks. *Cryptology ePrint Archive*, 2024.

[LXZ21] Tianyi Liu, Xiang Xie, and Yupeng Zhang. Zkcnn: Zero knowledge proofs for convolutional neural network predictions and accuracy. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, pages 2968–2985, 2021.

[PH23] Shahar Papini and Ulrich Haböck. Improving logarithmic derivative lookups using gkr. *IACR Cryptol. ePrint Arch.*, 2023:1284, 2023.

[Pip76] Nicholas Pippenger. On the evaluation of powers and related problems. In *17th Annual Symposium on Foundations of Computer Science (sfcs 1976)*, pages 258–263. IEEE Computer Society, 1976.

[QGYZ25] Wenjie Qu, Yanpei Guo, Yue Ying, and Jiaheng Zhang. Verfcnn, optimal complexity zksnark for convolutional neural networks. *Cryptology ePrint Archive*, 2025.

[QGZ26] Wenjie Qu, Yanpei Guo, and Jiaheng Zhang. Lightning, field-agnostic super-efficient polynomial commitment scheme. *Cryptology ePrint Archive*, 2026.

[QSL+25] Wenjie Qu, Yijun Sun, Xuanming Liu, Tao Lu, Yanpei Guo, Kai Chen, and Jiaheng Zhang. zkgpt: An efficient non-interactive zero-knowledge proof framework for llm inference. In *34st USENIX Security Symposium (USENIX Security 25)*, 2025.

[SCG+14] Eli Ben Sasson, Alessandro Chiesa, Christina Garman, Matthew Green, Ian Miers, Eran Tromer, and Madars Virza. Zerocash: Decentralized anonymous payments from bitcoin. In *2014 IEEE symposium on security and privacy*, pages 459–474. IEEE, 2014.

[Sch80] Jacob T Schwartz. Fast probabilistic algorithms for verification of polynomial identities. *Journal of the ACM (JACM)*, 27(4):701–717, 1980.

[Set20] Srinath Setty. Spartan: Efficient and general-purpose zksnarks without trusted setup. In *Annual International Cryptology Conference*, pages 704–737. Springer, 2020.

[SL20] Srinath Setty and Jonathan Lee. Quarks: Quadruple-efficient transparent zksnarks. *Cryptology ePrint Archive*, 2020.

[SLZ24] Haochen Sun, Jason Li, and Hongyang Zhang. zkllm: Zero knowledge proofs for large language models. *arXiv preprint arXiv:2404.16109*, 2024.

[STW24] Srinath Setty, Justin Thaler, and Riad Wahby. Unlocking the lookup singularity with lasso. In *Advances in Cryptology – EUROCRYPT 2024*, pages 180–209, 2024.

[Tha13] Justin Thaler. Time-optimal interactive proofs for circuit evaluation. In *Advances in Cryptology – CRYPTO 2013*, pages 71–89, 2013.

[TLI+23] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[YH24] Yibin Yang and David Heath. Two shuffles make a {RAM}: Improved constant overhead zero knowledge {RAM}. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 1435–1452, 2024.

[ZBK+22] Arantxa Zapico, Vitalik Buterin, Dmitry Khovratovich, Mary Maller, Anca Nitulescu, and Mark Simkin. Caulk: Lookup arguments in sublinear time. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, pages 3121–3134, 2022.

[ZGK+18] Yupeng Zhang, Daniel Genkin, Jonathan Katz, Dimitrios Papadopoulos, and Charalampos Papamanthou. vram: Faster verifiable ram with program-independent preprocessing. In *2018 IEEE Symposium on Security and Privacy (SP)*, pages 908–925. IEEE, 2018.

[ZGK+22] Arantxa Zapico, Ariel Gabizon, Dmitry Khovratovich, Mary Maller, and Carla Rafols. Baloo: Nearly optimal lookup arguments. *Cryptology ePrint Archive*, 2022.

[Zhu24] Michael Zhu. Engineering overview of lasso., 2024. URL: `https://github.com/a16z/jolt/blob/lasso/EngineeringOverview.md`.

[ZSG24] Yuncong Zhang, Shi-Feng Sun, and Dawu Gu. Efficient kzg-based univariate sum-check and lookup argument. In *IACR International Conference on Public-Key Cryptography*, pages 400–425. Springer, 2024.